
[26s] Software Development and Applications - Group 1 (Auto+SBT) (k_ESTM_009)

Variant 1

Final Report

Name: K Chandra Auditya

Matriculation No.: 100007450

Course: [26s] Software Development and Applications – Group
-1 (Auto+SBT) (k_ESTM_009)

Index

1. Question 1 — Movie Ratings (Lists and Strings)

Approach

Key Functions

Output

Explanation

2. Question 2 — Gradebook (Dictionaries and Sets)

Approach

Key Functions

Output

Explanation

3. Question 3 — Recipe Scaler (Functions)

Approach

Output

Explanation

5. Question 4 — Library (Classes and Inheritance)

Approach

Output

Explanation

6. Question 5 — Movie Review Extraction Pipeline (LLM + JSON)

Approach

Output

Real Transcript

Temperature Comparison

Explanation

7. Conclusion

Question 1 — Movie Ratings (Lists and Strings)

Approach

The task is to convert a single ratings string of the form "Title:rating, Title:rating, ..." into structured data and then compute several aggregates. My parser splits the string on commas to obtain the individual entries and splits each entry on the colon to separate the title from the rating. Whitespace is removed from both parts using the `strip()` method, so surrounding spaces around titles or numbers do not break parsing, and the rating is converted to an integer.

To improve efficiency and avoid redundant code, the `best_movie` function was refactored to directly call the existing `average_rating` function rather than calculating averages manually from scratch using a new dictionary.

Key Functions

- `parse_ratings(data)` — splits on commas, then colons; strips whitespace; converts rating to int.
- `average_rating(ratings, title)` — mean of one movie's ratings, rounded to 1 decimal; 0.0 if unknown.
- `best_movie(ratings)` — determines the unique titles and returns the title with the highest average utilizing the `average_rating` logic.
- `rating_counts(ratings)` — dictionary of rating counts per movie.

Output

```
PS A:\Software development> & "C:\Users\K chandra auditya\AppData\Local\Programs\Python\Python314\python.exe" "a:/Software development/Software_Development_and_Applications/Q1 (1).py"
=====
Q1: MOVIE RATINGS DEMO
=====

1. RAW DATA:
   Dune:8, Dune:9, Barbie:7, Dune:10, Barbie:9, Oppenheimer:9, Barbie:6

2. PARSED RATINGS (list of tuples):
   [('Dune', 8), ('Dune', 9), ('Barbie', 7), ('Dune', 10), ('Barbie', 9), ('Oppenheimer', 9), ('Barbie', 6)]

3. AVERAGE RATINGS:
   Dune: 9.0
   Barbie: 7.3
   Oppenheimer: 9.0
   Unknown Movie (Avatar): 0.0

4. BEST MOVIE (highest average):
   Dune

5. RATING COUNTS (per movie):
   Barbie: 3 ratings
   Dune: 3 ratings
   Oppenheimer: 1 ratings

=====
TEST: Extra whitespace handling
=====
Messy data:   Dune : 8 , Barbie : 9 , Dune : 10
Parsed:      [('Dune', 8), ('Barbie', 9), ('Dune', 10)]
```

Explanation

The main point of care was ensuring the rating was stored as an integer rather than a string and that whitespace was stripped from both the title and the number before conversion. Refactoring `best_movie` to use `average_rating` successfully streamlined the code and made it much cleaner.

Question 2 — Gradebook (Dictionaries and Sets)

Approach

The gradebook is a nested dictionary mapping each subject to a dictionary of students and their grades. The task strictly emphasizes using set operations over counting for `takes_all`. To accomplish this efficiently, the updated approach extracts the keys (students) from every subject, creates a list of sets, and unpacks them into Python's built-in `set`. `Intersection(*...)` method. These cleanly returns only the students present in every subject.

`student_average` collects a student's grades across all subjects and returns their mean rounded to two decimal places, or 0.0 if the student appears nowhere. `honor_roll` computes every student's average and returns, in alphabetical order, those whose average is less than or equal to the given limit (default 1.5).

Key Functions

- `subjects_of(student)` — set of subjects the student is enrolled in.
- `takes_all(grades)` — students present in every subject, via pure set intersection `set.intersection(*...)` (strictly no counting).
- `student_average(grades, student)` — mean grade across subjects, rounded to 2 decimals; 0.0 if absent.
- `honor_roll(grades, limit=1.5)` — alphabetically sorted students with average $\leq$ limit.

Output

```
PS A:\Software development> & "C:\Users\K chandra auditya\AppData\Local\Programs\Python\Python314\python.exe" "a:/Software development/Software_Development_and_Applications/Q2 (1).py"
anna subjects: {'math', 'art'}
anna average: 1.35
ben subjects: {'math', 'physics'}
ben average: 2.65
clara subjects: {'math', 'physics'}
clara average: 1.15
david subjects: {'physics', 'art'}
david average: 1.85
takes all: set()
unknown average: 0.0
honor roll: ['anna', 'clara']
```

Explanation

The key design choice was abandoning a for loop in `takes_all` and implementing it with `set.intersection(*student_sets)` to ensure complete adherence to the set operation rule. Care was also needed so that a student who appears in no subject returns an average of 0.0 rather than causing a division-by-zero error.

Question 3 — Recipe Scaler (Functions)

Approach

This task centers on flexible function signatures. The function signature `scale_recipe(name, servings, *ingredients, unit="g", **options)` combines required positional arguments, a variable number of ingredient tuples via `*ingredients`, a keyword-only argument `unit` with a default of `"g"`, and arbitrary keyword arguments via `**options` for cooking notes.

Each ingredient tuple contains an ingredient name and an amount for one serving. The function multiplies each amount by the number of servings, prints a scaled shopping list with the chosen unit appended to every amount, and prints any supplied options as cooking notes. If servings are less than one, it prints an error line and returns 0; otherwise, it returns a dictionary mapping each ingredient to its scaled amount.

Output

```
PS A:\Software development> & "C:\Users\K chandra auditya\AppData\Local\Programs\Python\Python314\python.exe" "a:/Software development/Software_Development_and_Applications/Q3 (1).py"

=== PASTA (servings: 4) ===
Shopping list:
  pasta: 400 g
  tomato sauce: 800 g
  cheese: 200 g

=== SMOOTHIE (servings: 2) ===
Shopping list:
  milk: 500 ml
  yogurt: 200 ml

=== CAKE (servings: 6) ===
Shopping list:
  flour: 1200 g
  sugar: 900 g
Cooking notes:
  oven: 180C
  time: 45min
ERROR: 'Pizza' needs servings >= 1, got 0
```

Explanation

The main subtlety was placing the `unit` parameter after `*ingredients` so that it becomes a keyword-only argument; otherwise, it would be consumed as another ingredient. The updated demonstration successfully tests new data across the four required cases: a simple recipe, a recipe with a custom unit, a recipe with keyword options, and the error case with `servings = 0`.

Question 4 — Library (Classes and Inheritance)

Approach

This task uses object-oriented programming. To ensure the code adheres to professional software development practices, Python Type Hints (: str, -> float, etc.) were integrated into the class structures.

The Book class stores a title, author, and year and implements __str__ for a readable representation, __eq__ so that two books are considered equal when their title and author match, and an age method returning how old a book is in a given year. EBook inherits from Book, adds a file size, overrides __str__ to include that size, and adds a download_seconds method. The Library class stores books in a list. Its add method relies on the custom __eq__ to ignore duplicates.

Output

```
PS A:\Software development> & "C:\Users\K chandra auditya\AppData\Local\Programs\Python\Python314\python.exe" "a:/Software development/Software_Development_and_Applications/Q4 (1).py"
Total books in library (__len__): 5
Orwell books (find_by_author): ['1984 by George Orwell (1949)', 'Animal Farm by George Orwell (1945)']
Oldest book (oldest): The Great Gatsby by F. Scott Fitzgerald (1925)
Age of 1984 in 2026 (age): 77
Download Dune at 10 Mbit/s (download_seconds): 2.0 s
Book string representation (__str__): 1984 by George Orwell (1949)
EBook string representation (__str__): Dune by Frank Herbert (1965) [2.5 MB]
```

Explanation

The download time calculation required attention to units. A file size given in megabytes must be converted to megabits before dividing by a link speed expressed in megabits per second, so the method computes (size_mb * 8) / mbit_per_s. Implementing __eq__ correctly was also important, since the Library.add method depends on it to detect and skip duplicates automatically.

Question 5 — Movie Review Extraction Pipeline (LLM + JSON)

Approach

This task builds a pipeline that makes a small local language model (qwen3-0.6b, served by LM Studio on port 1234) extract structured data from free-text movie reviews, and defends against the model returning invalid output.

- `ask_llm(prompt, system)` sends one chat completion with temperature 0 and returns only the message content.
- `build_prompt(text)` and `extract_review(text)` instruct the model to reply with a single JSON object.
- `check_review(d)` stringently validates the parsed object against the required schema, specifically checking that the rating is an integer and explicitly rejecting bool types.
- `extract_with_retry(text, max_attempts=3)` loops the validation and provides explicit error feedback to the model if it fails to adhere to the schema.
- Reviews List: The demo utilizes specific edge cases, including reviews containing ratings written out as words ("nine") and extremely negative reviews to test sentiment detection.

Output

```
PS A:\Software development> & "C:\Users\K chandra auditya\AppData\Local\Programs\Python\Python314\python.exe" "a:/Software development/Software_Development_and_Applications/Q5 (1).py"
```

Title	Rating	Sentiment
Dune	9	positive
Barbie	6	positive
Avengers	3	negative
Baahubali	9	positive
Toxic	1	negative

Positive: 3 Negative: 2

=====

BONUS: temperature comparison

=====

temperature=0.0 -> retries: 0, failures: 0

temperature=1.5 -> retries: 0, failures: 0

Real Transcript

The following is a real prompt and the raw reply returned by qwen3-0.6b during an actual run:

SYSTEM:

You extract structured data from movie reviews and reply with only JSON.

PROMPT:

Extract the movie review as a JSON object with exactly these keys: "title" (string), "rating" (integer 1-10), "sentiment" ("positive" or "negative"). Reply with ONLY the JSON object, nothing else.

Review: Baahubali still holds up. Gave it a nine.

RAW REPLY:

```
{"title":"Baahubali","rating":9,"sentiment":"positive"}
```

Bonus — Temperature Comparison

For the bonus task, the five reviews were run once at temperature 0 and once at temperature 1.5, counting the number of retries and outright failures.

Temperature	Retries	Failures
0.0	0	0
1.5	0	0

Observation: At temperature=0.0, the deterministic output ensures consistent adherence to the JSON schema. At temperature=1.5, the increased randomness can lead to formatting variations or non-standard keys, occasionally triggering retries in small LLMs (though the strict schema in this test case left little room for failure).

Explanation

The practical challenges in this task were environmental: ensuring the openAI package was installed correctly in the specific Python 3.14 environment and properly configuring LM Studio to run explicitly on the required <http://localhost:1234/v1> port to avoid WinError 10061 connection refused exceptions. Once the local server environment was aligned, the extraction and retry logic executed flawlessly.

Conclusion

All five tasks were implemented as independent Python files that run without errors under Python 3.11+ and demonstrate the required functionality in their demonstration blocks. The solutions exercise the core topics of the course: string and list processing, dictionaries and set operations, flexible function signatures with variable and keyword arguments, object-oriented programming with inheritance and special methods, and the construction of a validated, self-correcting language-model pipeline.

The optional bonus for Question 5 was completed. Only the standard library was used throughout, with the single permitted exception of the openAI package in Question 5.